

Conditionele statements

INFO

Lesinhoud

Controlestructuren: conditionele statements.

Doelstellingen

De student(e) kan:

- de basisprincipes van JavaScript-programmatie (variabelen en constanten, controlestructuren, functies, arrays, objecten, DOM en DOM-manipulatie, events en event handling) toepassen binnen een project;

▼ Inhoud

- [Controlestructuren](#)
- [Vergelijkingsoperators](#)
- [Conditionele statements](#)
 - [if-statement](#)
 - [if-else-statement](#)
 - [if-else if-else-statement](#)
- [Logische operators](#)
- [Conclusie](#)

Controlestructuren

Met controlestructuren bepalen we of een stukje code uitgevoerd wordt, en hoe vaak dit uitgevoerd wordt.

Soms willen we dat bepaalde code alleen wordt uitgevoerd als aan een bepaalde voorwaarde is voldaan. Andere keren willen we dat bepaalde code meerdere keren wordt uitgevoerd (herhaaldelijk wordt uitgevoerd) zolang aan een bepaalde voorwaarde is voldaan.

Er zijn twee soorten controlestructuren:

- **conditionele statements**: een stuk code wordt uitgevoerd **als** een bepaalde voorwaarde **true** is.
- **iteratieve statements**: een stuk code wordt herhaaldelijk uitgevoerd **zolang** een bepaalde voorwaarde **true** is.

In dit onderdeel worden de **conditionele statements** behandeld.

Vergelijkingsoperators

In eerdere onderdelen zijn de **toekenningoperator** en **rekenkundige operators** al aan bod gekomen. In dit onderdeel komen de **vergelijkingsoperators** en **logische operators** aan bod.

- **===** (strikt gelijk aan): Vergelijkt twee waarden en evalueert naar **true** als ze dezelfde waarde én hetzelfde type hebben.
- **==** (gelijk aan): Vergelijkt twee waarden en evalueert naar **true** als ze dezelfde waarde hebben.
- **!==** (strikt ongelijk aan): Vergelijkt twee waarden en evalueert naar **true** als ze niet dezelfde waarde hebben of niet van hetzelfde type zijn.
- **!=** (ongelijk aan): Vergelijkt twee waarden en evalueert naar **true** als ze niet dezelfde waarde hebben.

- `<` (kleiner dan): Evalueert naar `true` als de linkerwaarde kleiner is dan de rechterwaarde.
- `>` (groter dan): Evalueert naar `true` als de linkerwaarde groter is dan de rechterwaarde.
- `<=` (kleiner dan of gelijk aan): Evalueert naar `true` als de linkerwaarde kleiner is dan of gelijk is aan de rechterwaarde.
- `>=` (groter dan of gelijk aan): Evalueert naar `true` als de linkerwaarde groter is dan of gelijk is aan de rechterwaarde.

CONVENTIE

Gebruik `===` en `!==` in plaats van `==` en `!=` om te vergelijken. Op deze manier weet je zeker dat de waarden én types gelijk zijn, dit voorkomt onverwachte resultaten.

Bijvoorbeeld, `"10" === 10` evalueert naar `false`, omdat de string `"10"` en het number `10` wel dezelfde waarde hebben, maar niet hetzelfde type. Terwijl `"10" == 10` evalueert naar `true`, omdat de `==`-operator de types niet vergelijkt - de string `"10"` wordt omgezet naar een number `10` en dan vergeleken met `10`.

js

```
const getal1 = 5;
const getal2 = 10;

console.log(getal1 === getal2); // false, 5 (number) is
niet gelijk aan 10 (number)
console.log(getal1 !== getal2); // true, 5 (number) is
verschillend van 10 (number)
console.log(getal1 < getal2); // true, 5 (number) is
kleiner dan 10 (number)
console.log(getal1 > getal2); // false, 5 (number) is
niet groter dan 10 (number)
console.log(getal1 <= getal2); // true, 5 (number) is
kleiner dan of gelijk aan 10 (number)
console.log(getal1 >= getal2); // false, 5 (number) is
niet groter dan of gelijk aan 10 (number)
```

js

```
const getal1 = 10;
const getal2 = 10;

console.log(getal1 === getal2); // true, 10 (number) is
gelijk aan 10 (number)
console.log(getal1 !== getal2); // false, 10 (number) is
niet verschillend van 10 (number)
console.log(getal1 < getal2); // false, 10 (number) is
niet kleiner dan 10 (number)
console.log(getal1 > getal2); // false, 10 (number) is
niet groter dan 10 (number)
console.log(getal1 <= getal2); // true, 10 (number) is
kleiner dan of gelijk aan 10 (number)
console.log(getal1 >= getal2); // true, 10 (number) is
groter dan of gelijk aan 10 (number)
```

Het is ook mogelijk om variabelen van het type `string` met elkaar te vergelijken.

js

```
const naam1 = "John";
const naam2 = "Jane";

console.log(naam1 === naam2); // false, "John" (string)
is niet gelijk aan "Jane" (string)
console.log(naam1 !== naam2); // true, "John" (string) is
verschillend van "Jane" (string)
```

js

```
const naam1 = "John";
const naam2 = "John";

console.log(naam1 === naam2); // true, "John" (string) is
gelijk aan "John" (string)
console.log(naam1 !== naam2); // false, "John" (string)
is niet verschillend van "John" (string)
```

Conditionele statements

Conditionele statements worden gebruikt om code uit te voeren op basis van een bepaalde voorwaarde. Als de voorwaarde waar (`true`) is, wordt de code uitgevoerd. Als de voorwaarde niet waar (`false`) is, wordt de code overgeslagen.

`if` -statement

Een `if` -statement wordt gebruikt om een bepaald stuk code uit te voeren als de voorwaarde evalueert naar `true` . De syntax is als volgt:

```
const voorwaarde = true; //  
if (voorwaarde) {  
  // code die wordt uitgevoerd als de voorwaarde  
  evalueert naar true  
}
```

js

Als hetgeen tussen de haakjes `()` (de voorwaarde) evalueert naar `true` , dan wordt de code binnen de accolades `{}` uitgevoerd.

In bovenstaand voorbeeld is de variabele `voorwaarde` ingesteld op `true` , dus de code binnen de accolades wordt uitgevoerd.

```
const leeftijd = 19; // Momenteel hard-coded, later kan  
dit gevraagd worden aan de gebruiker via een prompt  
  
if (leeftijd >= 18) {  
  console.log("Je bent oud genoeg om te stemmen.");  
}
```

js

Als hetgeen tussen de haakjes `()` evalueert naar `true`, dan wordt de code binnen de accolades `{}` uitgevoerd. In andere woorden, als `leeftijd >= 18` evalueert naar `true`, dan wordt de code binnen de accolades `{}` uitgevoerd. `leeftijd >= 18` evalueert naar `true`, dus de code binnen de accolades `{}` wordt uitgevoerd. De code binnen de accolades `{}` zal een bericht tonen in de console: "Je bent oud genoeg om te stemmen."

In hetzelfde voorbeeld, waar een andere leeftijd is opgegeven:

```
js
const leeftijd = 17; // Momenteel hard-coded, later kan
dit gevraagd worden aan de gebruiker via een prompt

if (leeftijd >= 18) {
  console.log("Je bent oud genoeg om te stemmen.");
}
```

Hetgeen tussen de haakjes `()` wordt geëvalueerd, `leeftijd >= 18` waarbij `leeftijd` gelijk is aan `17`, evalueert naar `false`. De code binnen de accolades `{}` wordt dan niet uitgevoerd. Er wordt dus niets getoond in de console.

if - else -statement

In onderstaand voorbeeld wordt een `if`-statement getoond.

```
js
const leeftijd = 17; // Momenteel hard-coded, later kan
dit gevraagd worden aan de gebruiker via een prompt

if (leeftijd >= 18) {
  console.log("Je bent oud genoeg om te stemmen.");
}
```

Wat als we willen dat er ook iets gebeurt als de voorwaarde niet waar (`false`) is? Een optie is om een tweede `if`-statement te gebruiken.

```
js
const leeftijd = 17; // Momenteel hard-coded, later kan
dit gevraagd worden aan de gebruiker via een prompt

if (leeftijd >= 18) {
  console.log("Je bent oud genoeg om te stemmen.");
}

if (leeftijd < 18) {
  console.log("Je bent niet oud genoeg om te stemmen.");
}
```

Wanneer de leeftijd groter of gelijk is aan 18, dan zal de eerste `if`-statement uitgevoerd worden en zal de console tonen: "Je bent oud genoeg om te stemmen."

Wanneer de leeftijd kleiner is dan 18, dan zal de tweede `if`-statement uitgevoerd worden en zal de console tonen: "Je bent niet oud genoeg om te stemmen."

Er is echter een betere manier om dit te doen, namelijk door gebruik te maken van een `if-else`-statement.

```
js
const leeftijd = 17; // Momenteel hard-coded, later kan
dit gevraagd worden aan de gebruiker via een prompt

if (leeftijd >= 18) {
  console.log("Je bent oud genoeg om te stemmen.");
} else {
  console.log("Je bent niet oud genoeg om te stemmen.");
}
```

Wanneer de leeftijd groter of gelijk is aan 18, dan zal de `if`-statement evalueren naar `true` en zal de console tonen: "Je bent oud genoeg om te stemmen." Wanneer de leeftijd kleiner is dan 18 (kan ook gelezen worden als 'niet groter of gelijk aan 18'), dan zal de `if`-statement evalueren naar `false` en zal de code in de accolades `{}` van de `else`-statement uitgevoerd worden. De console zal dan tonen: "Je bent niet oud genoeg om te stemmen."

`if - else if - else`-statement

Stel dat we willen controleren of *nét* oud genoeg is (18), te jong (jonger dan 18) of oud genoeg (ouder dan 18), dan kunnen we dit ofwel doen door meerder `if`-statements te gebruiken.

Merk op dat de eerste `if`-statement nu enkel `> 18` controleert en niet `>= 18`.

```
js
const leeftijd = 18; // Momenteel hard-coded, later kan
dit gevraagd worden aan de

if (leeftijd > 18) {
  console.log("Je bent oud genoeg om te stemmen.");
}

if (leeftijd < 18) {
  console.log("Je bent te jong om te stemmen.");
}

if (leeftijd === 18) {
  console.log("Je bent net oud genoeg om te stemmen.");
}
```

De eerste `if`-statement controleert of de leeftijd groter is dan 18, dit evalueert naar `false`, dus de code binnen de accolades `{}` wordt niet

uitgevoerd.

De tweede `if`-statement controleert of de leeftijd kleiner is dan 18, dit evalueert naar `false`, dus de code binnen de accolades `{}` wordt niet uitgevoerd.

De derde `if`-statement controleert of de leeftijd gelijk is aan 18, dit evalueert naar `true`, dus de code binnen de accolades `{}` wordt uitgevoerd en de console toont: "Je bent net oud genoeg om te stemmen."

Dit werkt, maar er is opnieuw een betere manier om dit te doen, namelijk door gebruik te maken van een `if - else if - else`-statement.

```
js
const leeftijd = 18; // Momenteel hard-coded, later kan
dit gevraagd worden aan de gebruiker via een prompt

if (leeftijd > 18) {
  console.log("Je bent oud genoeg om te stemmen.");
} else if (leeftijd < 18) {
  console.log("Je bent te jong om te stemmen.");
} else {
  console.log("Je bent net oud genoeg om te stemmen.");
}
```

Wanneer de leeftijd groter is dan 18, dan zal de `if`-statement evalueren naar `true` en zal de console tonen: "Je bent oud genoeg om te stemmen.". De `else if`- en `else`-statements worden niet uitgevoerd.

Wanneer de leeftijd kleiner is dan 18, dan zal de eerste `if`-statement evalueren naar `false` en vervolgens wordt de `else if`-statement geëvalueerd. Deze evalueert naar `true` en de console toont: "Je bent te jong om te stemmen.". De `else`-statement wordt niet uitgevoerd.

Wanneer de leeftijd gelijk is aan 18, dan zal de eerste `if`-statement evalueren naar `false`, de `else if`-statement wordt uitgevoerd en

evalueert naar `false`, vervolgens wordt de `else`-statement uitgevoerd en de console toont: "Je bent net oud genoeg om te stemmen."

Logische operators

- `&&` (AND-operator): Evalueert naar `true` als beide voorwaarden `true` zijn.
- `||` (OR-operator): Evalueert naar `true` als ten minste één van de voorwaarden `true` is.
- `!` (NOT-operator): Keert de waarde van een voorwaarde om, evalueert naar `true` als de voorwaarde `false` is en vice versa.

```
const voorwaarde1 = true;  
const voorwaarde2 = false;
```

```
console.log(voorwaarde1 && voorwaarde2); // false,  
aangezien zowel voorwaarde1 als voorwaarde2 true moeten  
zijn voor de AND-operator om te evalueren naar true
```

```
console.log(voorwaarde1 || voorwaarde2); // true,  
aangezien één van de voorwaarden true moet zijn voor de  
OR-operator om te evalueren naar true
```

```
console.log(!voorwaarde1); // false, de NOT-operator  
keert de waarde van voorwaarde1 om  
console.log(!voorwaarde2); // true, de NOT-operator keert  
de waarde van voorwaarde2 om
```

```
console.log(voorwaarde1 && !voorwaarde2); // true,  
voorwaarde1 is true en voorwaarde2 is false, dus de NOT-  
operator keert de waarde van voorwaarde2 om naar true,  
dus voorwaarde1 en voorwaarde2 zijn beide true, dus de  
AND-operator evalueert naar true
```

js

Stel dat we willen controleren of iemand oud genoeg is om een feest binnen te gaan, maar enkel als ze meerderjarig zijn of als er een volwassene bij is.

```
js
const leeftijdPersoon1 = 17;
const leeftijdPersoon2 = 20;

if (leeftijdPersoon1 >= 18 || leeftijdPersoon2 >= 18) {
  console.log("Je mag het feest binnen.");
} else {
  console.log("Je mag het feest niet binnen.");
}
```

Wanneer de leeftijd van persoon 1 groter of gelijk is aan 18, zal de eerste voorwaarde evalueren naar `true` - de OR-operator evalueert naar `true` omdat ten minste één van de voorwaarden `true` is. De console toont: "Je mag het feest binnen."

In een andere situatie kan het zijn dat je alleen binnen mag als je zelf meerderjarig bent en als je niet te dronken bent.

```
js
const leeftijd = 19;
const nuchter = true;

if (leeftijd >= 18 && nuchter === true) {
  console.log("Je mag het feest binnen.");
} else {
  console.log("Je mag het feest niet binnen.");
}
```

De leeftijd is 19 dus groter of gelijk aan 18, de eerste voorwaarde evalueert naar `true`. De tweede voorwaarde evalueert naar `true` omdat `nuchter` gelijk is aan `true` - de vergelijking is dus `true === true`, wat evalueert naar `true`. De AND-operator evalueert naar `true`.

omdat beide voorwaarden `true` zijn. De console toont: "Je mag het feest binnen."

Wanneer je `true === true` als vergelijking hebt, is het voldoende om enkel de voorwaarde te gebruiken, zonder de expliciete vergelijking.

```
js
const leeftijd = 19;
const nuchter = true;

if (leeftijd >= 18 && nuchter) {
  console.log("Je mag het feest binnen.");
} else {
  console.log("Je mag het feest niet binnen.");
}
```

Dit doet exact hetzelfde als het vorige voorbeeld, maar is korter en leesbaarder.

Het kan ook zijn dat je bijhoudt of een persoon `teDronken` is of niet.

```
js
const leeftijd = 19;
const teDronken = false;

if (leeftijd >= 18 && !teDronken) {
  console.log("Je mag het feest binnen.");
} else {
  console.log("Je mag het feest niet binnen.");
}
```

Hier merk je het gebruik van de NOT-operator `!` op. 'Wanneer de persoon oud genoeg is en de persoon is niet te dronken, dan mag de persoon het feest binnen.'. `!teDronken` is hetzelfde als `teDronken === false`, maar korter.

Conclusie

Conditionele statements zijn een controlestructuur die wordt gebruikt om code wel of niet uit te voeren op basis van een bepaalde voorwaarde/conditie. In theorie kan dit met alleen `if`-statements worden gedaan, maar we hebben gezien dat het beter is om gebruik te maken van `if`, `if - else` en `if - else if - else`-statements. Dit zorgt ervoor dat de code geen dubbele condities bevat en dat de code leesbaarder is.

De voorwaarden kunnen bestaan uit vergelijkingen, zoals `leeftijd >= 18`, maar ook kunnen logische operators worden gebruikt om meerdere voorwaarden te combineren, zoals `leeftijd >= 18 && nuchter`.

Previous page
[Oefeningen](#)

Next page
[Oefeningen](#)